Person2 - Complete PySpark Source Code

Printed code companion for the executable .py and .ipynb. The executable files are authoritative; very long lines may be visually wrapped in this PDF.

```
"""LA2 PySpark source - Person2; assigned sites: COMO, CUPE, HOPB. Models: Decision Tree Regressor, Support Vector Regression (RBF), MLP
    # continued: Regressor (Neural Network)."""


# Environment and paths

from pathlib import Path
import time, math, json, os
import pandas as pd
import matplotlib.pyplot as plt
from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline as SklearnPipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
import warnings


PERSON = "Person2"
SITE_IDS = ['COMO', 'CUPE', 'HOPB']

def find_project_root():
    for candidate in [Path.cwd(), *Path.cwd().parents]:
        if (candidate / "dataset" / "raw").is_dir():
            return candidate
    raise FileNotFoundError(
        "Could not find dataset/raw. Start Jupyter from the extracted person folder, "
        "or update PROJECT_ROOT to point at that folder."
    )

PROJECT_ROOT = find_project_root()
RAW_DIR = PROJECT_ROOT / "dataset" / "raw"
OUTPUT_DIR = PROJECT_ROOT / "outputs"
FIGURE_DIR = OUTPUT_DIR / "figures"
TABLE_DIR = OUTPUT_DIR / "tables"
for folder in (OUTPUT_DIR, FIGURE_DIR, TABLE_DIR):
    folder.mkdir(parents=True, exist_ok=True)

spark = (
    SparkSession.builder
    .appName(f"LA2_{PERSON}_NEON_Nitrate_Forecasting")
    .config("spark.sql.session.timeZone", "UTC")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
print("Project root:", PROJECT_ROOT)
print("Selected sites:", SITE_IDS)
print("Spark version:", spark.version)



# Load original NEON CSVs

# Load only the original NEON NSW_15_minute measurement tables for the assigned sites.
site_paths = [str(RAW_DIR / site) for site in SITE_IDS]
raw = (
    spark.read
    .option("header", "true")
```

```
    .option("inferSchema", "false")
    .option("recursiveFileLookup", "true")
    .option("pathGlobFilter", "*NSW_15_minute*.csv")
    .csv(site_paths)
    .withColumn("source_path", F.input_file_name())
    .withColumn("siteID", F.regexp_extract(F.col("source_path"), r"/raw/([A-Z]{4})/", 1))
)
raw_count = raw.count()
print(f"Original measurement rows loaded: {raw_count:,}")
raw.printSchema()
raw.show(5, truncate=False)

# Quality-flag audit BEFORE filtering. NEON defines finalQF=0 as pass and finalQF=1 as fail.
quality_audit = (
    raw.withColumn("finalQF_int", F.col("finalQF").cast("int"))
       .groupBy("siteID", "finalQF_int")
       .agg(F.count(F.lit(1)).alias("row_count"))
       .orderBy("siteID", "finalQF_int")
)
quality_audit_pd = quality_audit.toPandas()
quality_audit_pd.to_csv(TABLE_DIR / "quality_flag_audit.csv", index=False)
display(quality_audit_pd)



# Quality filtering and 30-minute aggregation

# Parse timestamps and numeric fields. Keep nonnegative nitrate observations with finalQF=0.
parsed = (
    raw
    .withColumn("start_ts", F.to_timestamp("startDateTime"))
    .withColumn("end_ts", F.to_timestamp("endDateTime"))
    .withColumn("nitrate", F.col("surfWaterNitrateMean").cast("double"))
    .withColumn("nitrate_min_source", F.col("surfWaterNitrateMinimum").cast("double"))
    .withColumn("nitrate_max_source", F.col("surfWaterNitrateMaximum").cast("double"))
    .withColumn("expanded_uncertainty", F.col("surfWaterNitrateExpUncert").cast("double"))
    .withColumn("standard_error", F.col("surfWaterNitrateStdErMean").cast("double"))
    .withColumn("finalQF_int", F.col("finalQF").cast("int"))
)
quality_filtered = parsed.filter(
    (F.col("siteID").isin(SITE_IDS))
    & F.col("start_ts").isNotNull()
    & F.col("nitrate").isNotNull()
    & (F.col("nitrate") >= 0)
    & (F.col("finalQF_int") == 0)
)

# Remove duplicate site/timestamp rows after applying the quality rule so that a flagged duplicate
# cannot displace an accepted record. Retain one deterministic source path per site/timestamp.
dedup_window = Window.partitionBy("siteID", "start_ts").orderBy(F.col("source_path"))
clean_15m = (
    quality_filtered
    .withColumn("_duplicate_rank", F.row_number().over(dedup_window))
    .filter(F.col("_duplicate_rank") == 1)
    .drop("_duplicate_rank")
    .cache()
)
print(f"Accepted, deduplicated 15-minute observations: {clean_15m.count():,}")

# Aggregate 15-minute means into non-overlapping 30-minute bins. The timestamp used below is the
# END of each 30-minute bin, so a prediction made at time t uses only completed intervals <= t.
with_bins = clean_15m.withColumn("bin", F.window(F.col("start_ts"), "30 minutes"))
aggregated = (
    with_bins.groupBy("siteID", "bin")
```

```
    .agg(
        F.avg("nitrate").alias("nitrate_mean"),
        F.min("nitrate_min_source").alias("nitrate_min"),
        F.max("nitrate_max_source").alias("nitrate_max"),
        F.avg("expanded_uncertainty").alias("uncertainty_mean"),
        F.avg("standard_error").alias("std_error_mean"),
        F.count("nitrate").alias("valid_15min_count"),
    )
    .select("siteID", F.col("bin.end").alias("time"), "nitrate_mean", "nitrate_min",
            "nitrate_max", "uncertainty_mean", "std_error_mean", "valid_15min_count")
    .cache()
)
print(f"Available 30-minute intervals: {aggregated.count():,}")
aggregated.orderBy("siteID", "time").show(10, truncate=False)



# Lags and future targets

# Time features use the end time of the completed 30-minute interval.
features_df = (
    aggregated
    .withColumn("hour_value", F.hour("time") + F.minute("time") / F.lit(60.0))
    .withColumn("doy_value", F.dayofyear("time"))
    .withColumn("hour_sin", F.sin(F.lit(2.0 * math.pi) * F.col("hour_value") / F.lit(24.0)))
    .withColumn("hour_cos", F.cos(F.lit(2.0 * math.pi) * F.col("hour_value") / F.lit(24.0)))
    .withColumn("doy_sin", F.sin(F.lit(2.0 * math.pi) * (F.col("doy_value") - F.lit(1.0)) / F.lit(365.25)))
    .withColumn("doy_cos", F.cos(F.lit(2.0 * math.pi) * (F.col("doy_value") - F.lit(1.0)) / F.lit(365.25)))
    .withColumn("month", F.month("time").cast("double"))
    .drop("hour_value", "doy_value")
)

# Exact-time joins are used for lag/target features. A label is created only when an accepted
# aggregate exists exactly 24 hours or 7 days later; no interpolation is used to invent targets.
base_values = aggregated.select("siteID", "time", "nitrate_mean")

def add_shift_column(left_df, minutes, new_name, target=False):
    if target:
        # A future row at t+h is shifted back to t, placing its value beside the forecast origin.
        shifted = base_values.select(
            "siteID",
            (F.col("time") - F.expr(f"INTERVAL {minutes} MINUTES")).alias("time"),
            F.col("nitrate_mean").alias(new_name),
        )
    else:
        # A past value at t-h is shifted forward to t, becoming a predictor for the current row.
        shifted = base_values.select(
            "siteID",
            (F.col("time") + F.expr(f"INTERVAL {minutes} MINUTES")).alias("time"),
            F.col("nitrate_mean").alias(new_name),
        )
    return left_df.join(shifted, on=["siteID", "time"], how="left")

for minutes, name in [(30, "lag_30m"), (60, "lag_1h"), (1440, "lag_24h"), (10080, "lag_7d")]:
    features_df = add_shift_column(features_df, minutes, name, target=False)
features_df = add_shift_column(features_df, 1440, "target_24h", target=True)
features_df = add_shift_column(features_df, 10080, "target_7d", target=True).cache()

FEATURE_COLS = [
    "nitrate_mean", "lag_30m", "lag_1h", "lag_24h", "lag_7d",
    "nitrate_min", "nitrate_max", "uncertainty_mean", "std_error_mean",
    "valid_15min_count", "hour_sin", "hour_cos", "doy_sin", "doy_cos", "month"
]
print("Feature rows:", f"{features_df.count():,}")
```

```
print("24-hour labels available:", f"{features_df.filter(F.col('target_24h').isNotNull()).count():,}")
print("7-day labels available:", f"{features_df.filter(F.col('target_7d').isNotNull()).count():,}")
features_df.select("siteID", "time", "nitrate_mean", "lag_24h", "lag_7d", "target_24h", "target_7d").orderBy("siteID", "time").show(10,
    # continued: truncate=False)

site_coverage = (
    aggregated.groupBy("siteID")
    .agg(
        F.count(F.lit(1)).alias("aggregated_30min_intervals"),
        F.min("time").alias("first_interval_end_utc"),
        F.max("time").alias("last_interval_end_utc"),
        F.avg("nitrate_mean").alias("mean_nitrate_uM"),
        F.expr("percentile_approx(nitrate_mean, 0.5)").alias("median_nitrate_uM"),
        F.min("nitrate_mean").alias("min_nitrate_uM"),
        F.max("nitrate_mean").alias("max_nitrate_uM"),
    ).orderBy("siteID")
)
site_coverage_pd = site_coverage.toPandas()
site_coverage_pd.to_csv(TABLE_DIR / "site_coverage.csv", index=False)
display(site_coverage_pd)



# Exploratory visualisations

# Basic visualisations from the actual processed Spark dataset.
coverage_plot = site_coverage_pd.sort_values("siteID")
plt.figure(figsize=(8, 4.5))
plt.bar(coverage_plot["siteID"], coverage_plot["aggregated_30min_intervals"])
plt.title(f"{PERSON}: valid 30-minute intervals by site")
plt.xlabel("NEON site")
plt.ylabel("Intervals")
plt.tight_layout()
plt.savefig(FIGURE_DIR / "site_coverage.png", dpi=150)
plt.show()

# Daily average plot, reduced to one row per site/day before collecting to the notebook process.
daily_pd = (
    aggregated.withColumn("date", F.to_date("time"))
    .groupBy("siteID", "date")
    .agg(F.avg("nitrate_mean").alias("daily_mean_nitrate_uM"))
    .orderBy("date")
    .toPandas()
)
plt.figure(figsize=(10, 4.5))
for site in SITE_IDS:
    d = daily_pd[daily_pd["siteID"] == site]
    plt.plot(pd.to_datetime(d["date"]), d["daily_mean_nitrate_uM"], label=site, linewidth=0.8)
plt.title(f"{PERSON}: daily mean nitrate concentration")
plt.xlabel("Date (UTC)")
plt.ylabel("Nitrate (µmol/L)")
plt.legend(ncol=3)
plt.tight_layout()
plt.savefig(FIGURE_DIR / "daily_nitrate_time_series.png", dpi=150)
plt.show()

hist_pd = aggregated.select("siteID", "nitrate_mean").sample(False, 0.04, seed=42).toPandas()
plt.figure(figsize=(8, 4.5))
for site in SITE_IDS:
    vals = hist_pd.loc[hist_pd["siteID"] == site, "nitrate_mean"].dropna()
    if len(vals):
        plt.hist(vals, bins=45, alpha=0.35, density=True, label=site)
plt.title(f"{PERSON}: nitrate concentration distributions (sampled for plotting)")
plt.xlabel("Nitrate (µmol/L)")
```

```
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.savefig(FIGURE_DIR / "nitrate_distributions.png", dpi=150)
plt.show()



# Person-specific models and metrics

# Build strict per-site chronological train/test sets for both horizons.
def make_horizon_split(frame, target_col, horizon_sql):
    labeled = (frame.filter(F.col(target_col).isNotNull())
        .withColumn("label", F.col(target_col).cast("double"))
        .withColumn("target_time", F.col("time") + F.expr(f"INTERVAL {horizon_sql}")))
    order_w = Window.partitionBy("siteID").orderBy("time")
    count_w = Window.partitionBy("siteID")
    ranked = labeled.withColumn("_rn", F.row_number().over(order_w)).withColumn("_n", F.count(F.lit(1)).over(count_w))
    cutoff = ranked.groupBy("siteID").agg(F.max(F.when(F.col("_rn") <= F.floor(F.col("_n") * F.lit(0.80)),
        # continued: F.col("time"))).alias("cutoff_time"))
    split = ranked.join(cutoff, on="siteID", how="inner")
    helper_cols = ["_rn", "_n", "cutoff_time", "target_time"]
    train = split.filter((F.col("time") < F.col("cutoff_time")) & (F.col("target_time") < F.col("cutoff_time"))).drop(*helper_cols)
    test = split.filter(F.col("time") >= F.col("cutoff_time")).drop(*helper_cols)
    return train, test

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline as SklearnPipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings

NUMERIC_FEATURES = FEATURE_COLS
MODEL_FEATURES = NUMERIC_FEATURES + ["siteID"]
MAX_TRAIN_ROWS_PER_SITE = 1500
MAX_TEST_ROWS_PER_SITE = 2500

# The Spark DataFrame remains the source of truth for temporal splits. A bounded stratified
# sample is collected for driver-side estimators that are not all available in Spark MLlib.
def sample_by_site(frame, per_site, seed):
    sampled_parts = []
    for idx, site in enumerate(SITE_IDS):
        site_frame = frame.filter(F.col("siteID") == site)
        n_available = site_frame.count()
        if n_available > per_site:
            fraction = min(1.0, (2.0 * per_site) / float(n_available))
            piece = site_frame.sample(withReplacement=False, fraction=fraction, seed=seed + idx).limit(per_site)
        else:
            piece = site_frame
        sampled_parts.append(piece)
    out = sampled_parts[0]
    for part in sampled_parts[1:]:
        out = out.unionByName(part)
    return out

class LocalEstimatorPipeline:
    """Small adapter so any compatible estimator can use the common preprocessing."""
    def __init__(self, estimator):
        self.preprocessor = ColumnTransformer([
            ("numeric", SklearnPipeline([("imputer", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), NUMERIC_FEATURES),
            ("site", OneHotEncoder(handle_unknown="ignore", sparse_output=False), ["siteID"]),
        ], remainder="drop", sparse_threshold=0.0)
        self.estimator = estimator
```

```
    def fit(self, X, y):
        X_transformed = self.preprocessor.fit_transform(X)
        self.estimator.fit(X_transformed, y)
        return self
    def predict(self, X):
        return self.estimator.predict(self.preprocessor.transform(X))

def make_sklearn_pipeline(estimator):
    return LocalEstimatorPipeline(estimator)

def calculate_metrics(y_true, y_pred):
    return {"MAE": float(mean_absolute_error(y_true, y_pred)),
            "RMSE": float(mean_squared_error(y_true, y_pred) ** 0.5),
            "R2": float(r2_score(y_true, y_pred)) if len(y_true) > 1 and float(pd.Series(y_true).var()) > 0 else float("nan")}

def make_person_model_specs():
    return [
    ("Decision Tree Regressor", lambda: DecisionTreeRegressor(max_depth=12, min_samples_leaf=5, random_state=42)),
    ("Support Vector Regression (RBF)", lambda: SVR(kernel="rbf", C=10.0, epsilon=0.1, gamma="scale")),
    ("MLP Regressor (Neural Network)", lambda: MLPRegressor(hidden_layer_sizes=(64, 32), activation="relu", alpha=0.001,
        # continued: learning_rate_init=0.001, max_iter=80, early_stopping=True, random_state=42)),
]

MODEL_SPECS = make_person_model_specs()
MODEL_NAMES = [name for name, _ in MODEL_SPECS]
SCALING_MODEL_NAME = "Decision Tree Regressor"

model_rows = []
site_model_rows = []
scaling_train_pd_24 = None
scaling_test_pd_24 = None

for horizon, target_col, horizon_sql, seasonal_col in [
    ("24h", "target_24h", "1 DAY", "lag_24h"),
    ("7d", "target_7d", "7 DAYS", "lag_7d"),
]:
    train, test = make_horizon_split(features_df, target_col, horizon_sql)
    train_total, test_total = train.count(), test.count()
    sampled_train = sample_by_site(train, MAX_TRAIN_ROWS_PER_SITE, seed=42)
    sampled_test = sample_by_site(test, MAX_TEST_ROWS_PER_SITE, seed=142)
    train_pd = sampled_train.select("siteID", "time", "label", *NUMERIC_FEATURES).toPandas()
    test_pd = sampled_test.select("siteID", "time", "label", *NUMERIC_FEATURES).toPandas()
    if horizon == "24h":
        scaling_train_pd_24 = train_pd.copy()
        scaling_test_pd_24 = test_pd.copy()
    print(f"\nHorizon={horizon}; Spark train rows={train_total:,}; Spark test rows={test_total:,}; estimator training
        # continued: sample={len(train_pd):,}; evaluation sample={len(test_pd):,}")
    X_train, y_train = train_pd[MODEL_FEATURES], train_pd["label"].to_numpy()
    X_test, y_test = test_pd[MODEL_FEATURES], test_pd["label"].to_numpy()

    for model_name, model_factory in MODEL_SPECS:
        pipeline = make_sklearn_pipeline(model_factory())
        t0 = time.perf_counter()
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            pipeline.fit(X_train, y_train)
        fit_seconds = time.perf_counter() - t0
        t0 = time.perf_counter()
        predictions = pipeline.predict(X_test)
        predict_seconds = time.perf_counter() - t0
        overall = calculate_metrics(y_test, predictions)
        model_rows.append({"horizon": horizon, "model": model_name,
            "spark_train_rows": int(train_total), "spark_test_rows": int(test_total),
            "n_train_used": int(len(train_pd)), "n_test_evaluated": int(len(test_pd)),
```

```
            "fit_seconds_driver_estimator": fit_seconds, "predict_seconds_driver_estimator": predict_seconds,
            **overall})
        scored = test_pd[["siteID", "label"]].copy()
        scored["prediction"] = predictions
        for site in SITE_IDS:
            part = scored[scored.siteID == site]
            if len(part):
                site_metrics = calculate_metrics(part.label.to_numpy(), part.prediction.to_numpy())
                site_model_rows.append({"siteID": site, "horizon": horizon, "model": model_name,
                    "n_test": int(len(part)), **site_metrics})
        print(f"{model_name}: RMSE={overall['RMSE']:.4f}; MAE={overall['MAE']:.4f}; R2={overall['R2']:.4f}; fit={fit_seconds:.2f}s;
            # continued: predict={predict_seconds:.2f}s")

    # Naive baselines use exactly the same sampled holdout rows where each predictor exists.
    for baseline_name, baseline_col in [("Persistence (current value)", "nitrate_mean"), ("Seasonal naive (matching lag)", seasonal_col)]:
        baseline_pd = sampled_test.select("siteID", "label", baseline_col).toPandas().dropna(subset=[baseline_col])
        if baseline_pd.empty:
            continue
        pred = baseline_pd[baseline_col].to_numpy()
        overall = calculate_metrics(baseline_pd.label.to_numpy(), pred)
        model_rows.append({"horizon": horizon, "model": baseline_name,
            "spark_train_rows": int(train_total), "spark_test_rows": int(test_total),
            "n_train_used": int(len(train_pd)), "n_test_evaluated": int(len(baseline_pd)),
            "fit_seconds_driver_estimator": 0.0, "predict_seconds_driver_estimator": 0.0, **overall})
        for site in SITE_IDS:
            part = baseline_pd[baseline_pd.siteID == site]
            if len(part):
                bm = calculate_metrics(part.label.to_numpy(), part[baseline_col].to_numpy())
                site_model_rows.append({"siteID": site, "horizon": horizon, "model": baseline_name,
                    "n_test": int(len(part)), **bm})
        print(f"{baseline_name}: RMSE={overall['RMSE']:.4f}; MAE={overall['MAE']:.4f}; R2={overall['R2']:.4f}")

metrics_pd = pd.DataFrame(model_rows)
site_metrics_pd = pd.DataFrame(site_model_rows)
metrics_pd.to_csv(TABLE_DIR / "model_metrics_personal_models.csv", index=False)
site_metrics_pd.to_csv(TABLE_DIR / "site_model_metrics_personal_models.csv", index=False)
display(metrics_pd.sort_values(["horizon", "RMSE"]))
display(site_metrics_pd.sort_values(["siteID", "horizon", "RMSE"]))



# Execution-time and scalability analysis

# Sample-size scaling experiment for the primary algorithm assigned to this person.
# This measures estimator fit/predict time on the Spark-selected sample; Spark preprocessing is
# distributed above, while these particular estimators execute in the notebook Python process.
from sklearn.base import clone
scaling_rows = []
scaling_train = scaling_train_pd_24.copy()
scoring_pd = scaling_test_pd_24.head(min(3000, len(scaling_test_pd_24))).copy()
scaling_factory = dict(MODEL_SPECS)[SCALING_MODEL_NAME]
for frac in [0.25, 0.50, 1.00]:
    n = max(30, min(len(scaling_train), int(round(len(scaling_train) * frac))))
    part = scaling_train.sample(n=n, random_state=400 + int(frac * 100)) if n < len(scaling_train) else scaling_train
    pipeline = make_sklearn_pipeline(scaling_factory())
    t0 = time.perf_counter()
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        pipeline.fit(part[MODEL_FEATURES], part["label"])
    fit_seconds = time.perf_counter() - t0
    t0 = time.perf_counter()
    _ = pipeline.predict(scoring_pd[MODEL_FEATURES])
    predict_seconds = time.perf_counter() - t0
    scaling_rows.append({"training_fraction": frac, "training_rows": len(part), "scoring_rows": len(scoring_pd),
```

```
                        "fit_seconds": fit_seconds, "predict_seconds": predict_seconds,
                        "model": SCALING_MODEL_NAME})
    print(f"{frac:.0%}: train={len(part):,}; fit={fit_seconds:.2f}s; predict={predict_seconds:.2f}s")
scaling_pd = pd.DataFrame(scaling_rows)
scaling_pd.to_csv(TABLE_DIR / "scalability_results_personal_models.csv", index=False)
display(scaling_pd)
plt.figure(figsize=(7.5, 4.5))
plt.plot(scaling_pd["training_rows"], scaling_pd["fit_seconds"], marker="o", label="Fit")
plt.plot(scaling_pd["training_rows"], scaling_pd["predict_seconds"], marker="o", label="Predict")
plt.title(f"{PERSON}: estimator runtime vs sample size ({SCALING_MODEL_NAME})")
plt.xlabel("Training rows"); plt.ylabel("Wall-clock seconds (this machine)")
plt.legend(); plt.tight_layout(); plt.savefig(FIGURE_DIR / "runtime_scalability_personal_models.png", dpi=150); plt.show()



# Final result plots

# Comparison plot from this notebook's own model results.
ml_metrics = metrics_pd[metrics_pd["model"].isin(MODEL_NAMES)].copy()
ml_metrics["label"] = ml_metrics["horizon"] + " - " + ml_metrics["model"]
plt.figure(figsize=(10, max(5.5, 0.36 * len(ml_metrics))))
plt.barh(ml_metrics["label"], ml_metrics["RMSE"])
plt.xlabel("RMSE (µmol/L)")
plt.title(f"{PERSON}: 24-hour and 7-day model comparison")
plt.tight_layout(); plt.savefig(FIGURE_DIR / "model_rmse_comparison_personal_models.png", dpi=150); plt.show()
print("Outputs saved to:", OUTPUT_DIR)
print("Model metrics: ", TABLE_DIR / "model_metrics_personal_models.csv")
print("Site-level metrics: ", TABLE_DIR / "site_model_metrics_personal_models.csv")
print("Scalability metrics: ", TABLE_DIR / "scalability_results_personal_models.csv")
print("No model uses future target values as predictors. Read the final report's sampling note before interpreting model comparisons.")
```